Weekly Python Assignment 3 - Group 3 - S23

Instructions: For each exercise below, type the requested code into the cell below the exercise, then run your code to verify that it works correctly.

Color Code (nonfunctional in Google Colab):	**Relative Difficulty & Complexity**
- External File/Imported Module	- **Easy (E):** Covered during the week's lecture. Typically single-step exercise
- User-created Function	- **Medium (M):** Some combination with topics previously covered. Mostly double-step exercise
- Python/Imported Function	- **Hard (H):** Combination with topics previously covered. Multi-step exercise
- Name-specific Data Structure/Variable	

Breakdown: 2 (E), 5 (M), & 3 (H)

1(E). Create a list called list1 containing [10,9,8,7,6,5]. Print every other element, beginning at the second element in list1.

In [48]:
```
list1 = [10, 9, 8, 7, 6, 5]
for i in range (1, len(list1)):
    print(list1[i], end = " ")
# OR
print("")
print(list1[1:])
```

```
9 8 7 6 5
[9, 8, 7, 6, 5]
```

2(E). Print the list created in Ex. 01, list1, in reverse order.

In [49]:
```
for i in range (len(list1)-1, -1, -1):
    print(list1[i], end = " ")
```

```
5 6 7 8 9 10
```

3(M). Assign the first element from the list created in Ex. 01, list1, to a new variable. Remove the first element from list1, then print out the entire list. After that, insert it back into list1, to its original location, then print out the entire list.

Hint: Do not type the value of the element.

In [50]:
```
first_element = list1.pop(0)
print("list1, first element removed : ", end = "")
print(list1)
```

```
list1.insert(0, first_element) # index 0 is origianal postion
print(list1)
```

```
list1, first element removed : [9, 8, 7, 6, 5]
[10, 9, 8, 7, 6, 5]
```

4(M). Create a list called random_list containing [4,2,9,22,5,21,1,6,3,55,10,8,2,1,15,99]. Print all the values from random_list that are less than 10.

In [58]:
```
random_list = [4,2,9,22,5,21,1,6,3,55,10,8,2,1,15,99]
for i in random_list:
    if i < 10:
        print(i, end= " ")
```

```
4 2 9 5 1 6 3 8 2 1
```

5(M). Create a new list with sorted values from the list created in Ex. 04, random_list, without sorting the original random_list. Now sort the original random_list. Show that the two sorted lists are the same by testing them for equality and printing the results.

In [60]:
```
new_list = sorted(random_list) #sorted method (without sorting the original random_lis
random_list.sort()

if new_list == random_list:
    print("same")
else:
    print("Not same")

print("Sorted list", new_list)
print("Original list:", random_list)
```

```
same
Sorted list [1, 1, 2, 2, 3, 4, 5, 6, 8, 9, 10, 15, 21, 22, 55, 99]
Original list: [1, 1, 2, 2, 3, 4, 5, 6, 8, 9, 10, 15, 21, 22, 55, 99]
```

6(H). Ask the user to input a word, then ask the user to input a letter. Using either a For or a While loop and check if the inputted letter appears in the inputted word. If the inputted letter does not appear in the inputted word, print "Clear!". If it does appear in the inputted word, print "Trouble!". Test your code using "harmonica" and "f" as inputs.

In [68]:
```
word = input("Enter the word")
letter = input("Enter the letter")
sw = False

for i in word:
    if i == letter: # if the letter is in word. break the for statement and print outp
        sw = True
        break
    else:
        pass

if sw == True:
    print("Trouble!")
else:
    print("Clear!")
```

```
print("letter:", letter, " word:", word)
```

```
Clear!
letter: f  word: harmonica
```

7(M). The following lists are quality control samples listing the weights for 1-pound butter in a dairy plant:

- sample1=[16.00,15.94,16.05]
- sample2=[16.10,15.88,15.97]
- sample3=[16.12,16.09,16.14]
- sample4=[15.99,16.02,16.03]
- sample5=[16.02,16.01,16.04]

Create the 4 corresponding lists, using the data above: sample1,sample2,sample3, and sample4 Now, create a single list called trials with sample1,sample2, and sample3 as its elements. Then utilizing nested For loops, print all data values in trials with all values of a sample on one row. (e.g., all values of sample1 are on one row, all values of sample2 are on a different row,...)

In [2]:
```
sample1=[16.00,15.94,16.05]
sample2=[16.10,15.88,15.97]
sample3=[16.12,16.09,16.14]
sample4=[15.99,16.02,16.03]
# sample5=[16.02,16.01,16.04] The problem order to create 4 data sets.

trials = [sample1, sample2, sample3]
for i in trials:
    for j in i:
        print(j, end = " ")
    print("")
```

```
16.0 15.94 16.05
16.1 15.88 15.97
16.12 16.09 16.14
```

8(H). Create a new empty list called average_weights. Calculate the average weight for each of the samples in the list created in Ex. 14, trials, using For loops, but instead of printing them, append each value to average_weights. Then print the average_weights list. Finally, print the maximum and minimum values in the average_weights list using the format "The maximum weight is xxx, which is sample n" where xxx is the average weight and n is the corresponding sample number.

In [84]:
```
average_weights = []
for i in trials:
    avg = sum(i)/len(i)
    average_weights.append(avg)

maximum = max(average_weights)
minimum = min(average_weights)
idx_max = average_weights.index(maximum)
idx_min = average_weights.index(minimum)
```

```
print("The maximun weight is %f, which is sample %d" %(maximum, idx_max))
print("The minimun weight is %f, which is sample %d" %(minimum, idx_min))
```

```
The maximun weight is 16.116667, which is sample 2
The minimun weight is 15.983333, which is sample 1
```

9(M). Create the multiplication table up to 10*10 and store them in a list of lists. Then print the table, with the integers from 1 to 10 as both the first row and first column, and the rest of the table having the correct row*column value.

Hint: Remember that the option end="" can be used in the print function to turn off creation of a new line that is activated by default.

In [108…

```
lst = [[0 for _ in range(10)] for _ in range (10)] #10*10 table

lst[0] = [i for i in range(1, 11)] #fist row 1 - 10

for j in range (1,11): #first colum 1 - 10
    lst[j-1][0] = j

for x in range (1,10):
    for y in range (1,10):
        lst[x][y] = lst[0][x] * lst[y][0] #multiplication table

for a in lst: #print
    for b in a:
        print("%3d" %b, end = " ")
    print("")
```

```
  1   2   3   4   5   6   7   8   9  10
  2   4   6   8  10  12  14  16  18  20
  3   6   9  12  15  18  21  24  27  30
  4   8  12  16  20  24  28  32  36  40
  5  10  15  20  25  30  35  40  45  50
  6  12  18  24  30  36  42  48  54  60
  7  14  21  28  35  42  49  56  63  70
  8  16  24  32  40  48  56  64  72  80
  9  18  27  36  45  54  63  72  81  90
 10  20  30  40  50  60  70  80  90 100
```

10(H). Create a simple encoder by having the user input a word and an integer between 1 and 7. Add the integer value to the ASCII value of each letter of the inputted word to create an encoded word. Print the encoded word.

In [1]:

```
word = input("Enter a word to encode: ")
num = int(input("Enter a number between 1 and 7: "))

encoded_word = ""

for i in word:
    encoded_letter = chr(ord(i) + num)
    encoded_word += encoded_letter

print("Encoded word:", encoded_word)
```

```
Encoded word: v|jff
```